Національний університет «Полтавська політехніка імені Юрія Кондратюка»
Навчально-науковий інститут інформаційних технологій та робототехніки

Лабораторна робота №1

Звіт: результати виконання консольного застосунку та відповіді на контрольні запитання

Дисципліна: Технології на платформі .NET
Тема моделі: «Ігрова студія» (GameStudio): Employee → Programmer, Artist; GameProject; BugReport; generic CrudService<T>
Виконав: студент групи 402-ТН Желудков Віталій Віталійович
Перевірив: Тютюнник П. Б.
Середовище: .NET 8 (net8.0), C# 12

1. Результати виконання консольного застосунку GameStudio.Console

```
$ dotnet run --project GameStudio.Console
```

```
Лабораторна робота №1 — модель «Ігрова студія» (GameStudio)
Запуск: 21.10.2026 22:55

================================================================================================
1. Статичні поля, статичні конструктори та статичні методи
================================================================================================
Працівників створено до початку роботи: 0
Надбавка за рік стажу (статичне поле): 3%
Рушії, які підтримує студія: Unity, Unreal Engine, Godot, Roblox Studio
GameProject.IsEngineSupported("Godot")     = True
GameProject.IsEngineSupported("GameMaker") = False

================================================================================================
2. CRUD-сервіс працівників: Create та ReadAll
================================================================================================
  [подія ElementCreated] додано: Андрій Коваленко (Middle C#-програміст)
  [подія ElementCreated] додано: Олена Шевчук (Junior Luau-програміст)
  [подія ElementCreated] додано: Максим Бондар (3D-моделер)
  [подія ElementCreated] додано: Ірина Мельник (UI-художник)

Усі працівники (ReadAll):
  1. Middle C#-програміст         Андрій Коваленко       стаж: 5 р., зарплата: 51 750,00 грн
  2. Junior Luau-програміст       Олена Шевчук           стаж: 2 р., зарплата: 29 680,00 грн
  3. 3D-моделер                   Максим Бондар          стаж: 4 р., зарплата: 29 120,00 грн
  4. UI-художник                  Ірина Мельник          стаж: 3 р., зарплата: 26 160,00 грн
Кількість елементів у сервісі: 4
Працівників створено (статичний лічильник): 4

================================================================================================
3. Read — пошук працівника за Id
================================================================================================
Read(2805ccf8-c2aa-47c5-b248-8c71e836b3f5):
  Junior Luau-програміст       Олена Шевчук           стаж: 2 р., зарплата: 29 680,00 грн

================================================================================================
4. Update — зміна даних працівників
================================================================================================
Комітів у Олени: 42; підвищення грейду: так → Middle
  [подія ElementUpdated] оновлено: Олена Шевчук (Middle Luau-програміст)
Завершених 3D-моделей у Максима: 3 (бонус 250 грн за кожну)
  [подія ElementUpdated] оновлено: Максим Бондар (3D-моделер)
  [подія ElementUpdated] оновлено: Ірина Мельник (Аніматор)

Працівники після оновлення:
  1. Middle C#-програміст         Андрій Коваленко       стаж: 5 р., зарплата: 51 750,00 грн
  2. Middle Luau-програміст       Олена Шевчук           стаж: 2 р., зарплата: 44 520,00 грн
  3. 3D-моделер                   Максим Бондар          стаж: 4 р., зарплата: 29 870,00 грн
  4. Аніматор                     Ірина Мельник          стаж: 3 р., зарплата: 30 475,00 грн

================================================================================================
5. Remove — видалення працівника
================================================================================================
```

```
  [подія ElementRemoved] видалено: Ірина Мельник (Аніматор)

Працівники після видалення:
  1. Middle C#-програміст         Андрій Коваленко       стаж: 5 р., зарплата: 51 750,00 грн
  2. Middle Luau-програміст       Олена Шевчук           стаж: 2 р., зарплата: 44 520,00 грн
  3. 3D-моделер                   Максим Бондар          стаж: 4 р., зарплата: 29 870,00 грн

================================================================================================
6. Обробка винятків у CRUD-сервісі
================================================================================================
Read неіснуючого Id    → KeyNotFoundException: Елемент з Id 00000000-0000-0000-0000-000000000000 не знайдено.
Повторний Create       → InvalidOperationException: Елемент з Id 1d704619-1029-4a29-8792-24fdfc711597 уже
існує.
Повторний Remove       → KeyNotFoundException: Неможливо видалити: елемент з Id dcad65e5-1049-48c8-84bf-
13a605499c45 не знайдено.
MakeCommits(-5)        → ArgumentOutOfRangeException: Кількість комітів має бути додатною. (Parameter
'count')

================================================================================================
7. Методи розширення та пошук за умовою (делегати Func<T, bool> і Predicate<T>)
================================================================================================
Місячний фонд оплати праці (TotalMonthlyPayroll): 126 140,00 грн
Короткий опис (ToShortString): Андрій Коваленко (Middle C#-програміст)
Лише програмісти (Find з Func<Employee, bool>):
  1. Middle C#-програміст         Андрій Коваленко       стаж: 5 р., зарплата: 51 750,00 грн
  2. Middle Luau-програміст       Олена Шевчук           стаж: 2 р., зарплата: 44 520,00 грн
Зарплата понад 40 000 грн (Predicate<Employee>):
  - Андрій Коваленко (Middle C#-програміст): 51 750,00 грн
  - Олена Шевчук (Middle Luau-програміст): 44 520,00 грн

================================================================================================
8. Ігрові проєкти: власний делегат і події StatusChanged / BudgetExceeded
================================================================================================
  [подія ElementCreated] додано проєкт «Sky Pirates»
  [подія ElementCreated] додано проєкт «Pixel Farm»

  [подія StatusChanged] «Sky Pirates»: Concept → Prototype
  [подія StatusChanged] «Sky Pirates»: Prototype → Production
  [подія StatusChanged] «Pixel Farm»: Concept → Prototype
Витрачено 320 000 грн → використано бюджету: 64,0 %
  [подія BudgetExceeded] «Sky Pirates»: бюджет перевищено на 70 000,00 грн!
Витрачено ще 250 000 грн → використано бюджету: 114,0 %
  [подія StatusChanged] «Sky Pirates»: Production → Testing
  [подія StatusChanged] «Sky Pirates»: Testing → Released
Зміна статусу після релізу → InvalidOperationException: Проєкт "Sky Pirates" уже завершено (Released), статус
змінити не можна.
Непідтримуваний рушій       → ArgumentException: Рушій "GameMaker" не підтримується студією. (Parameter
'engine')

Усі проєкти (ReadAll):
  1. "Sky Pirates" (Action RPG, Roblox Studio) — Released, бюджет: 500 000 грн, витрачено: 570 000 грн, у
команді: 3
  2. "Pixel Farm" (Simulator, Godot) — Prototype, бюджет: 150 000 грн, витрачено: 0 грн, у команді: 1

================================================================================================
9. Баг-трекер: подія Fixed (вбудований делегат EventHandler)
================================================================================================
Усі баги:
  1. [Critical] Гравець провалюється крізь палубу корабля — відкрито
  2. [High    ] Не зберігається прогрес після виходу з гри — відкрито
  3. [Low     ] Друкарська помилка в головному меню — відкрито
Є відкриті критичні баги (HasOpenCriticalBugs): True

  [подія Fixed] виправлено баг: Гравець провалюється крізь палубу корабля
  [подія Fixed] виправлено баг: Друкарська помилка в головному меню
MarkAsFixed без виконавця → InvalidOperationException: Баг "Не зберігається прогрес після виходу з гри" ще
нікому не призначено.
Є відкриті критичні баги (HasOpenCriticalBugs): False
Відкриті баги (Find з умовою !IsFixed):
  1. [High    ] Не зберігається прогрес після виходу з гри — відкрито

================================================================================================
10. Додаткове завдання: Save / Load — серіалізація у JSON-файли
================================================================================================
Save → data/employees.json (828 байт)
Save → data/projects.json (656 байт)
Save → data/bugs.json (1061 байт)
```

```
Фрагмент файлу employees.json:
  [
    {
      "$type": "programmer",
      "PrimaryLanguage": "C#",
      "Grade": "Middle",
      "CommitsCount": 0,
      "Id": "1d704619-1029-4a29-8792-24fdfc711597",
      "FullName": "Андрій Коваленко",
      "HireDate": "2021-03-15T00:00:00",
      "BaseSalary": 30000
    },
    {
  ...

Load → працівників: 3, проєктів: 2, багів: 3
Завантажені працівники (тип об'єкта збережено завдяки полю "$type"):
  Programmer | Middle C#-програміст         Андрій Коваленко       стаж: 5 р., зарплата: 51 750,00 грн
  Programmer | Middle Luau-програміст       Олена Шевчук           стаж: 2 р., зарплата: 44 520,00 грн
  Artist     | 3D-моделер                   Максим Бондар          стаж: 4 р., зарплата: 29 870,00 грн
Завантажені проєкти:
  1. "Sky Pirates" (Action RPG, Roblox Studio) — Released, бюджет: 500 000 грн, витрачено: 570 000 грн, у
команді: 3
  2. "Pixel Farm" (Simulator, Godot) — Prototype, бюджет: 150 000 грн, витрачено: 0 грн, у команді: 1
Завантажені баги:
  1. [Critical] Гравець провалюється крізь палубу корабля — виправлено
  2. [High    ] Не зберігається прогрес після виходу з гри — відкрито
  3. [Low     ] Друкарська помилка в головному меню — виправлено
Ідентифікатори збігаються з оригіналом: True; фонд оплати праці збігається: True
Load неіснуючого файлу → FileNotFoundException: Файл з даними не знайдено.

================================================================================================
Підсумок
================================================================================================
Усього створено об'єктів Employee (разом із завантаженими з файлу): 8
Роботу програми завершено.
```

2. Відповіді на контрольні запитання

1. Що таке класи та структури? Яка різниця між класом та структурою? Яка різниця між value та reference типами?

Клас — це користувацький тип даних, «креслення» для створення об'єктів. Він описує стан (поля, властивості) і поведінку (методи, конструктори, події). Клас є **посилальним (reference) типом**. У лабораторній це `Employee`, `Programmer`, `GameProject`, `CrudService<T>`.

Структура (`struct`) теж об'єднує дані й методи, але є **значущим (value) типом**. Її використовують для невеликих, переважно незмінних наборів даних. Вбудовані структури є і в нашій моделі: `Guid Id`, `DateTime HireDate`, `decimal Budget`.

	Клас (class)	**Структура (struct)**
Категорія типу	reference-тип	value-тип
Присвоєння `b = a`	копіюється посилання, обидві змінні вказують на один об'єкт	копіюється все значення, отримуємо незалежну копію
Наслідування	може наслідуватися від іншого класу і бути базовим	не наслідується і не може бути базовою (неявно sealed, походить від `System.ValueType`), але може реалізовувати інтерфейси
Значення за замовчуванням	`null`	усі поля нульові (`0`, `false`…), `null` лише як `Nullable<T>`
Абстрактні/віртуальні члени, фіналізатор	можна	не можна

Value-типи (`int`, `double`, `decimal`, `bool`, `char`, `enum`, `struct`, `DateTime`, `Guid`) зберігають дані безпосередньо у змінній: локальні змінні — зазвичай у стеку, поля — всередині об'єкта, який їх містить. **Reference-типи** (`class`, `string`, масиви, інтерфейси, делегати, `object`) зберігають у змінній лише посилання на об'єкт у керованій купі (heap), яку очищає збирач сміття (GC).

```
public struct Point { public int X; public int Y; }

Point a = new Point { X = 1 };
Point b = a;          // копія значення
b.X = 99;             // a.X як і раніше = 1

Programmer p1 = new Programmer("Андрій", 30000m, "C#", ProgrammerGrade.Middle);
Programmer p2 = p1;   // копія посилання
p2.MakeCommits(5);    // p1.CommitsCount теж = 5 — це той самий об'єкт
```

Саме тому в консольному застосунку зміни, зроблені методами `olena.MakeCommits()` і `olena.Promote()`, одразу видно в об'єкті, що зберігається в `Dictionary` CRUD-сервісу.

2. Що таке абстрактний клас? Яка різниця між звичайним та абстрактним класом?

Абстрактний клас позначається словом `abstract` і слугує лише базовим класом для нащадків. Створити його екземпляр через `new` не можна. Він може містити як звичайні члени з реалізацією (поля, властивості, конструктори, методи), так і **абстрактні члени** без реалізації. Кожен неабстрактний нащадок зобов'язаний перевизначити їх через `override`.

```
public abstract class Employee : IEntity
{
    protected Employee(string fullName, decimal baseSalary, DateTime? hireDate = null) { ... }
    public int GetYearsOfExperience(DateTime? onDate = null) { ... }   // звичайний метод
    public abstract decimal CalculateMonthlySalary();                   // абстрактний метод
    public abstract string Position { get; }                            // абстрактна властивість
}

public class Programmer : Employee
{
    public override decimal CalculateMonthlySalary() { ... }  // обов'язкова реалізація
}
// var e = new Employee();  — помилка компіляції
```

Звичайний клас	Абстрактний клас
можна створювати об'єкти (`new`)	об'єкти створити не можна, лише об'єкти нащадків
не може містити абстрактних членів	може містити абстрактні члени, які нащадки мусять реалізувати
може бути `sealed` чи `static`	не може бути `sealed` чи `static`

Абстрактний клас зручний, коли в нащадків є спільний код і дані (у нас Id, ПІБ, дата найму, надбавка за стаж), але частина поведінки різна (зарплата програміста й художника рахується по-різному).

3. Що таке інтерфейс? Яка різниця між абстрактним класом та інтерфейсом?

Інтерфейс (`interface`) — це контракт: перелік методів, властивостей, подій та індексаторів, які зобов'язаний реалізувати клас (або структура). Інтерфейс описує, *що* об'єкт уміє робити, але не *як*. Члени інтерфейсу за замовчуванням `public`. Починаючи з C# 8, інтерфейс може містити реалізацію за замовчуванням і статичні члени, але полів екземпляра й конструкторів у ньому немає.

```
public interface IEntity { Guid Id { get; set; } }

public interface ICrudService<T>
{
    void Create(T element);  T Read(Guid id);  IEnumerable<T> ReadAll();
    void Update(T element);  void Remove(T element);
}

public interface IPersistentCrudService<T> : ICrudService<T>   // наслідування інтерфейсів
{
    void Save(string filePath);  void Load(string filePath);
}

public class CrudService<T> : IPersistentCrudService<T> where T : class, IEntity { ... }
```

Обмеження `where T : IEntity` гарантує, що будь-який тип у сервісі має `Id`. Тому той самий `CrudService<T>` працює і з `Employee`, і з `GameProject`, і з `BugReport`.

Абстрактний клас	Інтерфейс
клас може мати лише **один** базовий клас	клас може реалізувати **багато** інтерфейсів
може мати поля (стан) і конструктори	полів екземпляра й конструкторів немає
члени можуть мати будь-які модифікатори доступу	члени за замовчуванням `public`
описує відношення «є» (Programmer *є* Employee) і дає спільну реалізацію	описує здатність, «уміє» (сутність *має* Id, сервіс *уміє* CRUD)
структура не може від нього наслідуватися	структура може реалізувати інтерфейс

4. Що таке наслідування? Які є модифікатори доступу у мові C#?

Наслідування — принцип ООП, за яким похідний клас отримує члени базового класу, може додавати нові й перевизначати (`override`) віртуальні чи абстрактні. Це дає повторне використання коду й поліморфізм: з об'єктами `Programmer` і `Artist` можна працювати через змінну типу `Employee`, а викликається «правильна» версія `CalculateMonthlySalary()`. У C# клас має лише один базовий клас, але може реалізовувати кілька інтерфейсів. Усі типи зрештою наслідуються від `System.Object`, тому ми можемо перевизначити `ToString()`.

```
public class Artist : Employee            // Artist наслідує Employee
{
    public Artist(string fullName, decimal baseSalary, ArtSpecialization specialization,
                  string mainTool, DateTime? hireDate = null)
        : base(fullName, baseSalary, hireDate)   // виклик конструктора базового класу
    { ... }
}
```

Ключові слова: `virtual` (метод можна перевизначити), `override` (перевизначення), `abstract` (обов'язково перевизначити), `sealed` (заборона подальшого наслідування чи перевизначення), `base` (звернення до членів базового класу).

Модифікатори доступу:

Модифікатор	Звідки доступний	Приклад у лабораторній
`public`	звідусіль	властивості моделі, методи CRUD
`private`	лише всередині цього класу (за замовчуванням для членів класу)	`_createdCount`, `_storage`, `private set` у `Spent`
`protected`	у цьому класі та його нащадках	конструктори `Employee`, `ApplyExperienceBonus()`
`internal`	у межах однієї збірки (проєкту); за замовчуванням для класів верхнього рівня	—
`protected internal`	у межах збірки **або** в нащадках з інших збірок	—
`private protected`	лише в нащадках **і** лише в межах тієї ж збірки	—

5. Що таке статичні поля та методи? Яка різниця між звичайними та статичними полями та методами?

Статичні члени (`static`) належать самому **типу**, а не окремому об'єкту. Статичне поле існує в одному екземплярі, спільному для всіх об'єктів класу. Статичний метод викликається через ім'я класу і не має доступу до `this` та нестатичних членів. **Статичний конструктор** не має параметрів і модифікаторів доступу. Він викликається автоматично один раз, перед першим створенням об'єкта або першим зверненням до статичного члена.

```
public abstract class Employee
{
    private static int _createdCount;                    // статичне поле: одне на всіх
    public static readonly decimal ExperienceBonusPerYear;

    static Employee()                                    // статичний конструктор
    {
        _createdCount = 0;
        ExperienceBonusPerYear = 0.03m;
    }

    protected Employee(...) { ...; _createdCount++; }    // кожен об'єкт збільшує лічильник
    public static int GetCreatedCount() => _createdCount; // статичний метод
}

Employee.GetCreatedCount();              // виклик через ім'я класу
GameProject.IsEngineSupported("Godot"); // ще один статичний метод
```

Звичайні (екземплярні) члени	Статичні члени
у кожного об'єкта своя копія поля	одна копія на весь клас, спільна для всіх об'єктів
існують, поки існує об'єкт	існують увесь час роботи програми
виклик через об'єкт: `olena.Promote()`	виклик через клас: `Employee.GetCreatedCount()`
мають доступ до `this`, статичних і нестатичних членів	немає `this`, доступні лише статичні члени

Також: константа `const` (у нас `Artist.BonusPerArtwork`) неявно статична, а **статичний клас** (`ModelExtensions`) містить лише статичні члени, його не можна створити чи наслідувати. Саме в такому класі оголошуються методи розширення.

6. Що таке делегати? Які види делегатів існують?

Делегат — це типобезпечне посилання на метод (або кілька методів) із певною сигнатурою. Делегат можна зберегти у змінній, передати як параметр і викликати. Делегати лежать в основі подій, лямбда-виразів і методів-зворотних викликів (callback). Делегати є reference-типами (походять від `System.MulticastDelegate`) і можуть бути **групові (multicast)**: через `+=` до одного делегата можна додати кілька методів, і вони викличуться по черзі.

Подія (`event`) — це делегат, захищений класом-власником: викликати подію може лише сам клас, а зовнішній код тільки підписується (`+=`) і відписується (`-=`).

Види делегатів:

- **Власні (користувацькі)** — оголошуються словом `delegate`:
  `public delegate void ProjectStatusChangedHandler(GameProject project, ProjectStatus oldStatus, ProjectStatus newStatus);`
  На ньому побудована подія `GameProject.StatusChanged`.
- **`Action<…>`** — вбудований узагальнений делегат для методу, що **нічого не повертає** (0–16 параметрів). У нас: `event Action<T> ElementCreated` у CRUD-сервісі та `event Action<GameProject, decimal> BudgetExceeded`.
- **`Func<…, TResult>`** — вбудований делегат для методу, що **повертає значення** (останній параметр типу — результат). У нас: `Find(Func<T, bool> predicate)`.
- **`Predicate<T>`** — метод, що приймає `T` і повертає `bool`. У нас: `Predicate<Employee> earnsMoreThan40K`.
- **`EventHandler`** / **`EventHandler<TEventArgs>`** — стандартні делегати для подій із сигнатурою `(object? sender, EventArgs e)`. У нас: подія `BugReport.Fixed`.

Екземпляр делегата створюють, передаючи йому ім'я методу, анонімний метод або **лямбда-вираз**:

```
skyPirates.StatusChanged += OnProjectStatusChanged;                    // іменований метод
employeeService.ElementCreated += e => Console.WriteLine(e.FullName);  // лямбда-вираз
Func<Employee, bool> isProgrammer = e => e is Programmer;
var programmers = employeeService.Find(isProgrammer);
```